Auditing AI Agent Decisions in a $3^9 = 19{,}683$-State Trinary Cognitive Space

RUNZEAI — Independent Researcher

Version 1.2 — 2026

Abstract

Autonomous LLM agents increasingly execute multi-step tasks with tool access, yet their decisions remain difficult to audit: logs record *what* happened but not whether each step was well-founded, authorized, and consistent with the trajectory around it. We present a symbolic audit layer that encodes every agent decision step as a point in a trinary nine-dimensional cognitive space of $3^9 = 19{,}683$ states, and derives audit verdicts from transparent, configurable rules over that space. Each step is mapped to a nine-dimensional balanced-ternary vector (each dimension in $\{-1, 0, +1\}$) through an explicit signal-to-dimension mapping; the vector is enumerated bijectively to an integer key $\text{idx} \in [0, 19682]$, with the all-neutral center state at $\text{idx} = 9841$ and a signed balance value $\text{bal} \in [-9841, +9841]$. Audit rules operate on two structures: red-line dimensions (permission overreach → BLOCK; missing evidence or high risk → ALERT) and trajectory jumps (adjacent-step Hamming distance > 4 → WATCH), yielding a four-level verdict lattice OK / WATCH / ALERT / BLOCK with per-verdict dispositions. The layer is implemented as an open-source MCP server (`agent-audit-mcp`) with three tools, requires no external services, and is fully deterministic. We verify exhaustively that the state enumeration is a bijection over all 19,683 states, that repeated audits of identical input produce byte-identical reports (1,000 repetitions × 4 scenarios), and that four synthetic scenarios spanning all four verdict levels produce the expected verdicts. The layer is positioned as a deterministic, enumerable complement to semantic guardrails: it does not judge content, but it makes every decision step locatable, comparable, and reproducible — properties that audit reports and compliance workflows require.

Keywords: AI agent auditing; trinary logic; nine-dimensional cognitive space; balanced ternary; Hamming distance; Model Context Protocol; deterministic symbolic systems; agent safety

1. Introduction

Large language model (LLM) agents that interleave reasoning with tool calls — as popularized by ReAct-style scaffolds [6], Toolformer-style tool-use training [8], and Reflexion-style verbal self-improvement [7] — are moving from demos to production. As autonomy grows, so does the blast radius of a single bad step: an agent with tool access can touch systems, move data, and make commitments on a schedule no human reviewed in real time.

Observability tooling answers *what happened*: traces, spans, and event logs reconstruct the execution stream. Auditing, however, must answer a different question: *was each step well-founded?* Concretely, an operator needs to know whether the agent

1. **stayed within its authority** (no permission overreach),
2. **grounded high-risk conclusions in evidence** (no unsupported sign-off),
3. **kept its outputs traceable** (every claim sourceable), and

4. **behaved consistently along its own trajectory** (no unexplained state jumps).

These are properties of the *decision process*, not of the *event stream*. Logs alone cannot express them, because a log entry is an event, while an audit finding is a judgment about a step relative to rules and to its neighbors.

This paper builds that judgment layer on a discrete symbolic substrate. We map each agent decision step onto a point in the **trinary nine-dimensional cognitive space** — nine observation dimensions, each taking one of three states {−1, 0, +1}, giving 3^9 = 19,683 enumerable states [1]. On top of this space we define a small, transparent rule system that produces four-level verdicts (OK / WATCH / ALERT / BLOCK) with concrete dispositions, and we ship the whole thing as a deterministic, local, open-source MCP server so that any agent orchestrator can call it as a log hook, a pre-execution gate, or a post-hoc report generator.

Contributions.

1. A formal mapping from agent decision steps to the 3^9 = 19,683-state trinary cognitive space, including a bijective enumeration (idx), a signed balance value (bal), and a Hamming distance over states (Section 3–4).
2. A trajectory-level audit rule system with red-line dimensions, jump detection, a verdict lattice with defined precedence, and per-verdict dispositions (Section 5).
3. An open-source reference implementation (`agent-audit-mcp` , MIT) exposing the layer as three MCP tools, with no external dependencies and full determinism (Section 6).
4. Verification: exhaustive bijection check over all 19,683 states, byte-identical determinism over 1,000 repetitions × 4 scenarios, and four scenario audits covering all four verdict levels (Section 7).

Scope. The layer is symbolic and rule-based. It does not read natural-language content and does not replace semantic guardrails [5]; it assumes that an orchestrator (or a signal-extraction component) can supply structured per-step signals. Its job is to make the *shape* of every decision locatable, comparable, and reproducible.

---

2. Related Work

Agent reasoning and tool use. ReAct [6] interleaves reasoning traces with actions; Toolformer [8] teaches models to invoke APIs; Reflexion [7] adds verbal self-feedback across episodes. These works expand what agents can do. Auditability is orthogonal: a more capable agent needs a stronger audit substrate, not a weaker one.

Guardrails and alignment. Constitutional AI [5] and downstream moderation stacks operate on content semantics — they ask whether an output is harmful. Our layer asks whether a *step* was authorized, evidenced, and trajectory-consistent. The two are complements: semantic guardrails judge content; this layer judges process geometry.

Agent observability. Distributed tracing and LLM-specific tracing frameworks record the event stream (prompts, completions, tool calls, latencies). They are necessary but insufficient for audit findings: there is no verdict, no rule, and no notion of step-to-step consistency. Our layer consumes the same signal family and adds the missing judgment.

Three-valued and symbolic formalisms. Three-valued logic dates to Łukasiewicz [3] and underpins SQL's NULL semantics. The trinary paradigm (yin / zhong / yang, i.e., −1 / 0 / +1) is classical in the *Yijing* [9] and has been re-formalized in prior work of this line of research as the Trinary Nine-Dimensional Cognitive Coordinate System [1]: 3^9 = 19,683 states, equivalence classes, and a semantic bridge. The present paper builds the *agent-audit application layer* on that substrate.

Formal verification of agents. Model checking and theorem proving offer strong guarantees but require formal system models that production LLM agents do not have. Our approach deliberately sits between: a discrete, enumerable state space with transparent rules — weaker guarantees than verification, but applicable to black-box LLM agents today.

---

3. Formal Framework: The Trinary Nine-Dimensional Cognitive Space

3.1 Definition

Let D = {−1, 0, +1}^9 be the set of nine-dimensional trinary vectors. Each vector v = (v_1, ..., v_9) is a *cognitive state*: v_i = +1 reads "strong / above baseline" on dimension i, v_i = 0 reads "neutral", v_i = −1 reads "weak / deficient". The state space has cardinality

D	**= 3^9 = 19,683.**

We write states as nine-symbol strings over the alphabet {⊖, 0, ⊕}, e.g. `0⊖0⊖000⊖0`.

3.2 Enumeration Bijection

Define the enumeration key

idx(v) = Σ_{i=1}^{9} (v_i + 1) · 3^{9-i}.

This is the standard base-3 encoding of the digit vector (v_i + 1) ∈ {0,1,2}^9, hence a bijection between D and the integer interval [0, 19682]. The inverse idx → v recovers each digit by repeated division. Two derived quantities matter for auditing:

- **Center state.** v = *(0,...,0) maps to idx* = (19683 − 1)/2 = **9841**. It is the unique state in which every dimension is neutral.
- **Balance value.** bal(v) = idx(v) − 9841 ∈ [−9841, +9841], with bal(v*) = 0. bal is a signed scalar summary: positive means the state leans "above baseline" overall, negative means "deficient" overall.

Proposition 1. idx is a bijection D ↔ [0, 19682]; consequently every decision step has exactly one address, and two steps are identical iff their addresses are equal.

Verification. We exhaustively round-tripped all 19,683 states (Section 7.1): for every idx, index→vector→index returns idx, and the 19,683 recovered vectors are pairwise distinct.

3.3 Distance

For two states v, w ∈ D, the Hamming distance

h(v, w) = Σ_{i=1}^{9} [v_i ≠ w_i] ∈ [0, 9]

counts differing dimensions. In a decision trajectory, h measures how much the agent's cognitive posture changed between two steps — the discrete analogue of behavioral drift.

3.4 Properties That Matter for Auditing

- **Enumerable.** Every step has a unique integer address; audit findings can cite idx instead of prose.
- **Comparable.** Any two steps (or any two agents, or any two time windows) can be compared via h, and summarized via bal.

- **Reproducible.** The encoding is a pure function: identical signals always yield the identical state, on any machine, forever.
- **Local.** The engine is pure arithmetic with zero external dependencies; the decision trajectory never leaves the auditor's machine.

---

4. Encoding Agent Decision Steps

4.1 Signal Schema

A decision step is described by a *signal object*. Two fields are required — `tool_calls` (non-negative integer) and `intent` (one of `retrieve` / `reason` / `produce` / `execute` / `confirm`). The remaining fields are optional with safe neutral defaults, so an orchestrator can start with minimal instrumentation and enrich over time. Field names accept both snake_case and camelCase and are canonicalized idempotently.

Field	Type	Default	Audit meaning
`tool_calls`	int ≥ 0	*required*	number of tool invocations in the step
`intent`	enum	*required*	dominant semantic intent of the step
`context_ok`	bool	true	step is aligned with its declared context
`strong_dep`	bool	false	step strongly depends on an external role/tool
`evidence`	int ≥ 0	0	count of supporting evidence items
`confidence`	float ∈ [0,1]	0.5	self-reported confidence
`elapsed_ratio`	float > 0	1.0	step duration / baseline duration
`unauthorized`	bool	false	step requested an out-of-scope capability
`sensitive`	bool	false	step touches a sensitive surface
`citations`	int ≥ 0	0	traceable citations in the output
`traceable`	bool	true	citations are resolvable to sources
`risk_hit`	bool	false	step matched a high-risk rule
`value_add`	bool	false	step produced compliant value-added output
`regress`	bool	false	step moved the session backward
`jump_over`	bool	false	step skipped intermediate stages

4.2 Dimension Mapping

The nine dimensions and their operationalization (default thresholds in parentheses):

Dim	Name	Operational rule
v1	Structure (task structure)	⊖ if `tool_calls` ≤ 0; ⊕ if ≥ `toolHi` (3); else 0
v2	Function (step intent)	⊖ if `retrieve`; ⊕ if `produce` / `execute` / `confirm`; else 0
v3	Relation (context relevance)	⊖ if not `context_ok`; ⊕ if `strong_dep`; else 0
v4	Attribute (information sufficiency)	⊖ if `evidence` < `eviLo` (2); ⊕ if ≥ `eviHi` (5) or `confidence` ≥ `highConf` (0.8); else 0
v5	Time (step cadence)	⊖ if `elapsed_ratio` > `tSlow` (3.0); ⊕ if < `tFast` (0.5); else 0
v6	Context (permission boundary)	⊖ if `unauthorized`; ⊕ if `sensitive`; else 0
v7	Causality (decision evidence chain)	⊖ if `citations` ≤ 0; ⊕ if ≥ `citeHi` (3) and `traceable`; else 0
v8	Value (risk level)	⊖ if `risk_hit`; ⊕ if `value_add`; else 0
v9	Evolution (state transition)	⊖ if `regress`; ⊕ if `jump_over`; else 0

The mapping is intentionally shallow and inspectable: each rule is a one-line threshold on one or two signals. Dimensions v6, v7, v8 are designated **red-line dimensions** (Section 5.1).

4.3 Worked Example

Input signals:

```
{ "tool_calls": 2, "intent": "produce", "evidence": 4,
  "citations": 3, "traceable": true, "risk_hit": true }
```

Applying the mapping: v1 = 0 (1 ≤ 2 < 3), v2 = +1 (`produce`), v3 = 0 (context ok, no strong dep), v4 = 0 (2 ≤ 4 < 5, confidence 0.5 < 0.8), v5 = 0 (ratio 1.0), v6 = 0 (authorized, not sensitive), v7 = +1 (citations 3 ≥ 3 and traceable), v8 = −1 (`risk_hit`), v9 = 0. Output:

```
{ "vector": [0,1,0,0,0,0,1,-1,0], "vector_str": "0⊕0000⊕⊖0",
  "idx": 12034, "bal": 2193, "flags": ["RED:v8高风险"],
  "semantic": "strong/above baseline: step intent, decision evidence chain; weak/deficient: risk level" }
```

Note the audit-relevant asymmetry the encoding exposes: the step is *well-intentioned and well-cited* (v2, v7 positive) yet *high-risk* (v8 = −1) — a combination a scalar "risk score" would blur.

4.4 Configurability

All thresholds live in a single `RuleConfig` record and are overridable via JSON or environment variables (prefix `NINE_AUDIT_`, e.g. `NINE_AUDIT_JUMP=3`). Thresholds are defaults calibrated for illustration; a deployment re-calibrates them against its own logs. The rules remain transparent to the operator — there is no hidden model inside the auditor.

---

5. Trajectory Audit: Red Lines, Jumps, and the Verdict Lattice

5.1 Red-Line Dimensions

Three dimensions carry hard semantics. When a step's state has v_i = −1 on a red-line dimension, the step raises a flag:

Dimension	Flag	Severity
v6 (permission boundary)	`RED:v6越权` (overreach)	BLOCK
v7 (decision evidence chain)	`RED:v7依据缺失` (missing evidence)	ALERT
v8 (risk level)	`RED:v8高风险` (high risk)	ALERT

v6 = −1 means the step requested capability outside its scope: the session is blocked, rolled back, and re-authorized. v7 = −1 or v8 = −1 means the step made (or nearly made) a consequential commitment without a traceable basis: automation stops and a human compliance review is required.

5.2 State-Jump Detection

For each adjacent pair of steps (j−1, j) we compute h. If h > `jump` (default 4), the pair is recorded as a jump with its distance. A jump is not a violation — it is a *review trigger*: a conservative step followed by a maximally aggressive step (h up to 9) is exactly the shape of a state-jump escape, where an agent abandons its prior posture mid-trajectory.

5.3 Verdict Function

For a session trajectory t = (s_1, …, s_n) with encoded states v^(1), …, v^(n):

```
Verdict(t) = BLOCK  if ∃ k: v6(v^(k)) = -1
           = ALERT  else if ∃ k: v7(v^(k)) = -1 or v8(v^(k)) = -1
           = WATCH  else if ∃ j ∈ {2..n}: h(v^(j-1), v^(j)) > jump
           = OK     otherwise
```

The verdicts form a severity lattice **OK < WATCH < ALERT < BLOCK**; the function returns the maximum severity triggered, with the precedence order as written. The lattice is what makes the report actionable: one field, four levels, unambiguous routing.

5.4 Dispositions

Each verdict carries a fixed disposition list, so the audit output is directly consumable by an orchestrator or a human operator:

Verdict	Disposition
BLOCK	halt execution; roll back to the step before the overreach; retry after authorization
ALERT	stop automation; route to human compliance review; supplement evidence and traceable citations; regenerate with higher confidence
WATCH	human review of each jump point (with step indices and distances)
OK	continue execution

---

6. Implementation: `agent-audit-mcp`

6.1 Architecture

The reference implementation is a small Python package (≥ 3.10) with five modules:

- `engine` — the pure-math core: enumeration, balance, Hamming, rendering (Section 3).
- `signals` — the signal schema: canonicalization, validation, safe defaults (Section 4.1).

- `rules` — `RuleConfig` and the dimension mapping (Section 4.2).
- `audit` — step encoding and session-level trajectory audit with the verdict function (Section 5).
- `server` — the MCP interface (Section 6.2).

The core is a pure function pipeline `signals → state → trajectory → verdict`, with no I/O, no clocks, and no randomness.

6.2 MCP Interface

The layer is exposed as an MCP server (`name: agent-audit`) with three tools:

Tool	Input	Output
`audit_agent_step`	one signal object	`vector` , `vector_str` , `idx` , `bal` , `semantic` , `flags`
`audit_agent_session`	list of signal objects + optional `session_id`	`verdict` , `reasons` , `trajectory` , `red_lines` , `jumps` , `actions` , `state_count`
`encode_state`	a nine-symbol vector **or** an idx (plus optional second vector)	`vector` , `idx` , `bal` , `is_center` , optional `hamming`

The server runs over stdio for local connectors or streamable HTTP for remote ones. Once registered, an orchestrator (or a conversational agent operator) calls the tools inline — e.g. "audit this session" — and receives the JSON report.

6.3 Determinism, Locality, Testing

Every code path is deterministic: the same signals produce byte-identical reports (verified in Section 7.2). Nothing leaves the host: there is no network call, no model call, no telemetry. The test suite comprises 31 tests covering the engine (enumeration, balance, distance), the signal validator (required fields, ranges, canonicalization), the rule mapping, and the session auditor (verdict precedence, red lines, jumps, dispositions). The package is published on PyPI (`agent-audit-mcp` , MIT) and mirrored on GitHub.

---

7. Evaluation

We evaluate three claims: (E1) the state space machinery is correct (bijection, center, balance, distance); (E2) the auditor is deterministic; (E3) the verdict function routes the four intended scenario shapes to the four verdict levels. All numbers below come from running the released package.

7.1 E1 — State-Space Correctness

- **Exhaustive bijection.** For every idx ∈ [0, 19682]: `index_to_vector(idx)` → `vector_to_index(·)` returns idx (19,683/19,683 round-trips pass), and the recovered vectors are pairwise distinct (19,683/19,683).
- **Center and balance.** `index_to_vector(9841)` = (0,...,0); `bal(9841)` = 0; extremes `bal(0)` = −9841 and `bal(19682)` = +9841.
- **Distance sanity.** h(v_min, v_max) = 9 (all dimensions differ); h(v, v) = 0 for all v.

7.2 E2 — Determinism

Each of the four scenarios (Section 7.3) was audited 1,000 times. In all 4,000 runs the serialized report was byte-identical to the first run — including idx, bal, flags, jumps, and verdict.

7.3 E3 — Scenario Coverage

Four synthetic two-step sessions were designed to instantiate the four verdict levels (advisory-style traces; the layer itself is domain-agnostic). Real engine output:

Scenario	Steps	States (vector_str → idx)	Red lines	Jumps (h)	Verdict	Expected
A · normal advisory	2	`0⊖0000000` → 7654; `0⊕0⊕00⊕⊕0` → 12283	none	none	**OK**	OK
B · high-risk full-position advice	2	`0⊖0⊖000⊖0` → 7408; `⊖⊕⊖⊖⊕0⊖⊖⊕` → 4565	v8; v7+v8	(1→2, 6)	**ALERT**	ALERT
C · unauthorized query + stock pick	2	`0⊖⊖⊖0⊖⊖00` → 6646; `⊕⊕⊖00⊖0⊖0` → 17830	v6+v7; v6+v8	(1→2, 5)	**BLOCK**	BLOCK
D · state-jump escape	2	`0⊖0000000` → 7654; `⊕⊕⊕⊕⊕00⊕⊕` → 19646	none	(1→2, 7)	**WATCH**	WATCH

4/4 scenarios produce the expected verdict. The scenario set also exercises precedence: B triggers both ALERT-level flags and a jump (h = 6), and ALERT correctly dominates WATCH; C triggers BLOCK, which dominates everything.

7.4 Test Suite

31 unit and integration tests pass, covering: enumeration round-trips on boundary and random states; balance extremes; signal validation (missing required fields, out-of-range values, naming canonicalization); the nine dimension rules at their threshold boundaries; verdict precedence; red-line flagging; jump detection; and the JSON shape of both audit tools.

7.5 What This Evaluation Does Not Show

The scenarios are synthetic and hand-designed; they demonstrate the rule system's routing, not its calibration on production logs. Threshold defaults (e.g. `jump` = 4) are starting points. A deployment should measure flag rates on real traces and re-calibrate via `RuleConfig`. We also do not evaluate signal-extraction quality: the layer trusts its input signals, and garbage-in-garbage-out applies to the *signals*, while the encoding and verdicts remain exact functions of them.

---

8. Discussion

8.1 Intended Role

The layer is a *process-geometry* instrument. Three integration patterns follow directly from the verdict lattice:

1. **Pre-execution gate.** Before an agent executes a consequential step, encode the planned step; BLOCK stops the pipeline before the action exists.
2. **Log hook.** Wrap the orchestrator so every completed step is encoded as it happens; the session report is available at any time for an operator or a downstream compliance job.
3. **Post-hoc report.** Replay a stored session's signals to regenerate the exact audit report — reproducibility is a property of the encoding, so the report is a stable artifact that can be attached to an incident review.

Because the verdict is an enumerable function of the signals, audit findings can be cited by idx (e.g. "step 2 entered state 4565, a v7/v8 red-line state") in a way that is unambiguous, compact, and machine-checkable.

8.2 Limitations

- **Signal dependence.** The layer is exact given its signals; obtaining faithful signals (especially `unauthorized`, `risk_hit`, `traceable`) is the deployment's responsibility and the main engineering cost.
- **Coarse granularity.** Three states per dimension is a deliberate trade: 19,683 states are enumerable and explainable, but a finer grid (e.g. five levels) would trade enumerability for resolution. The current design favors auditability.
- **Rule simplicity.** Red lines and the jump threshold are transparent but shallow; they encode *process* hygiene, not content correctness.
- **Single-session scope.** Jumps are computed within one trajectory; cross-session drift is left to future work (the balance value and Hamming distance are available primitives for it).
- **No semantic judgment.** A step can be perfectly "geometric" (all zeros, no flags) and still be substantively wrong. The layer narrows the audit surface; it does not close it.

8.3 Position Against Semantic Guardrails

Semantic guardrails are probabilistic, content-aware, and opaque; this layer is deterministic, content-blind, and transparent. They fail differently, which is the argument for using both: the symbolic layer catches process violations (overreach, missing evidence, posture jumps) exactly and reproducibly, while semantic layers catch content violations probabilistically. The symbolic layer's verdicts can be written directly into compliance reports; the semantic layer's scores cannot, without additional calibration narrative.

---

9. Conclusion

We presented a symbolic audit layer for AI agent decisions built on the trinary nine-dimensional cognitive space: every decision step becomes one of 3^9 = 19,683 enumerable states, and a small transparent rule system over that space yields four-level verdicts (OK / WATCH / ALERT / BLOCK) with concrete dispositions. The reference implementation `agent-audit-mcp` is open-source, local, deterministic, and exposed as MCP tools, with exhaustive verification of the state-space bijection, byte-identical determinism, and full scenario coverage of the verdict lattice.

Agents will keep getting more autonomous. "Autonomous" must not mean "unsupervised": giving every decision an enumerable, comparable, reproducible cognitive coordinate is one of the foundations for landing AI safely — and, unlike most safety machinery, it can be audited itself.

References

[1] RUNZEAI. *Trinary Nine-Dimensional Cognitive Coordinate System: A Locatable and Generalizable Symbolic Cognition Layer*. Zenodo, 2026. DOI: 10.5281/zenodo.23437930.

[2] Anthropic. *Model Context Protocol*. https://modelcontextprotocol.io (accessed 2026).

[3] Łukasiewicz, J. "O logice trójwartościowej" (On three-valued logic). *Ruch Filozoficzny*, 1920.

[4] Hamming, R. W. "Error Detecting and Error Correcting Codes." *The Bell System Technical Journal*, 29(2):147–160, 1950.

[5] Bai, Y., Kadavath, S., Kundu, S., et al. "Constitutional AI: Harmlessness from AI Feedback." arXiv:2212.08073, 2022.

[6] Yao, S., Zhao, J., Yu, D., et al. "ReAct: Synergizing Reasoning and Acting in Language Models." arXiv:2210.03629, 2022.

[7] Shinn, N., Labash, B., Gopinath, A., et al. "Reflexion: Language Agents with Verbal Reinforcement Learning." arXiv:2303.11366, 2023.

[8] Schick, T., Dwivedi-Yu, J., Dessì, R., et al. "Toolformer: Language Models Can Teach Themselves to Use Tools." arXiv:2302.04761, 2023.

[9] *I Ching (Yijing)*, trans. R. Wilhelm, Routledge & Kegan Paul, 1951 (CBA Foundation translation).

Artifact

- This paper (Zenodo): DOI 10.5281/zenodo.23444061
- Package (PyPI, MIT): https://pypi.org/project/agent-audit-mcp/
- Source (GitHub): https://github.com/q1z2q3-debug/agent-audit-mcp
- Companion article: "Giving Your AI Agent a Black Box: Auditing Every Decision in a 3^9 = 19,683-State Cognitive Space", dev.to, 2026.

> Compliance note: the scenario traces in Section 7.3 are synthetic illustrations in an advisory style; the audit layer is domain-agnostic. This paper describes decision-audit encoding and verdicts only and does not constitute investment advice or securities consulting services.